Departamento de Electrónica, Sistemas e Informática

Big Data

Spring 2026

Lab 05: Writing data into PostgreSQL tables

Profesor: Pablo Camarillo Ramirez

Alumno: Bryan Edgardo Romo Gonzalez

Create SparkSession

In [1]:
```
!ls /opt/spark/work-dir/notebooks/jars/postgresql-42.7.8.jar
```

```
/opt/spark/work-dir/notebooks/jars/postgresql-42.7.8.jar
```

In [2]:
```
from spark_utils import SparkUtils

su = SparkUtils("Examples on storage solutions with PosgreSQL",
                "spark://spark-master:7077",
```

```
                spark_jars="/opt/spark/work-dir/notebooks/jars/postgresql-42.7.8.jar")
su.spark
```

```
WARNING: Using incubator modules: jdk.incubator.vector
26/03/16 21:59:35 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes w
here applicable
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
```

Out[2]:

SparkSession - in-memory

SparkContext

Spark UI

Version	`v4.0.1`
Master	`spark://spark-master:7077`
AppName	`Examples on storage solutions with PosgreSQL`

Create DataFrames

In [14]:

```
import pyspark.sql.functions as F
movies_columns = [("MovieID", "int"),
                  ("Title", "string"),
                  ("Genre", "string"),
                  ("ReleaseYear", "int"),
                  ("ReleaseDate_Str", "string"),
                  ("Country", "string"),
                  ("BudgetUSD", "float"),
                  ("US_BoxOfficeUSD", "float"),
                  ("Global_BoxOfficeUSD", "float"),
                  ("Opening_Day_SalesUSD", "float"),
                  ("One_Week_SalesUSD", "float"),
                  ("IMDbRating", "float"),
                  ("RottenTomatoesScore", "float"),
                  ("NumVotesIMDb", "int"),
                  ("NumVotesRT", "int"),
```

```
                  ("Director", "string"),
                  ("LeadActor", "string")]

movies_schema = SparkUtils.generate_schema(movies_columns)

movies_date_str_df = (su.spark.read
                .option("header", "true")
                .schema(movies_schema)
                .csv("/opt/spark/work-dir/data/movies/"))

movies_df = movies_date_str_df.withColumn("ReleaseDate", F.to_date("ReleaseDate_Str", "dd-MM-yyyy")).drop("ReleaseDate_Str")
#movies_df.show()
```

In [18]:
```
# Generamos la transformacion
sdf_revenue = (movies_df
       .groupBy("Country")
       .agg(F.sum("Global_BoxOfficeUSD").alias("total_revenue"))
       .orderBy(F.col("total_revenue"), ascending=False)
       .limit(10))

sdf_revenue.show()
```

```
[Stage 30:==============================>                          (1 + 1) / 2]
+-----------+--------------------+
|    Country|       total_revenue|
+-----------+--------------------+
|        USA|1.942559027841888...|
|         UK|1.424604528102836E12|
|      India| 1.39429636295725E12|
|     Canada|1.106603414509304...|
|      China|8.301483620182422E11|
|  Australia|8.277308239177812E11|
|     France|8.253183622110156E11|
|    Germany|5.507426223985312E11|
|      Japan|5.431587094455937...|
|South Korea|2.780330088543281E11|
+-----------+--------------------+
```

In [6]:
```
jdbc_url = "jdbc:postgresql://postgres-iteso:5432/postgres"
table_name = "grouped_movies"

movies_df.write \
    .format("jdbc") \
    .option("url", jdbc_url) \
    .option("dbtable", table_name) \
    .option("user", "postgres") \
    .option("password", "Admin@1234") \
    .option("driver", "org.postgresql.Driver") \
    .mode("overwrite") \
    .save()

print("DataFrame successfully written into a PosgreSQL DB !")
```

```
[Stage 10:>                                                         (0 + 1) / 1]
DataFrame successfully written into a PosgreSQL DB !
```

Write data to a PostgreSQL DB

In [16]:
```
jdbc_url = "jdbc:postgresql://postgres-iteso:5432/postgres"
table_name = "top_revenue_by_country"

sdf_revenue.write \
    .format("jdbc") \
    .option("url", jdbc_url) \
    .option("dbtable", table_name) \
    .option("user", "postgres") \
    .option("password", "Admin@1234") \
    .option("driver", "org.postgresql.Driver") \
    .mode("overwrite") \
    .save()

print(f"DataFrame escrito exitosamente en la tabla '{table_name}'")
```

```
[Stage 26:==============================>                          (1 + 1) / 2]
DataFrame escrito exitosamente en la tabla 'top_revenue_by_country'
```

Read data to a PostgreSQL DB

In [17]:
```
db_properties = {
    "user": "postgres",
    "password": "Admin@1234",
    "driver": "org.postgresql.Driver"
}

# Leer la tabla desde la DB
df_final = (su.spark.read.jdbc(url=jdbc_url, table=table_name, properties=db_properties))

# Mostrar esquema e información final
df_final.printSchema()
df_final.show(truncate=False)
```

```
root
 |-- Country: string (nullable = true)
 |-- total_revenue: double (nullable = true)

+-----------+---------------------+
|Country    |total_revenue        |
+-----------+---------------------+
|USA        |1.9425590278418883E13|
|UK         |1.424604528102836E12 |
|India      |1.39429636295725E12  |
|Canada     |1.1066034145093047E12|
|China      |8.301483620182422E11 |
|Australia  |8.277308239177812E11 |
|France     |8.253183622110156E11 |
|Germany    |5.507426223985312E11 |
|Japan      |5.4315870944559375E11|
|South Korea|2.780330088543281E11 |
+-----------+---------------------+
```

In [19]:
```
su.spark.stop()
```

In [ ]: